

## CODE PYTHON DE LA SIMULATION

```
import numpy as np
import matplotlib.pyplot as plt

# ===== PARAMÈTRES DE MARCHÉ ET PRODUIT =====
S0 = 4200.0          # Spot initial Euro Stoxx 50
r = 0.03             # Taux d'intérêt sans risque (OIS)
q = 0.035            # Dividend yield
sigma = 0.18         # Volatilité implicite 3Y
T = 3.0              # Maturité en années
n_steps = 12         # Fréquence trimestrielle (12 dates)
dt = T / n_steps     # Pas de temps (0.25 an)
N_paths = 200_000    # Nombre de trajectoires (réduit pour l'annexe)

# Paramètres du Payoff
coupon_q = 0.015     # 1.5% trimestriel
autocall_barrier = 1.00 # 100%
coupon_barrier = 0.80 # 80%
capital_barrier = 0.60 # 60%
notional = 100       # Nominal normalisé à 100

# ===== FONCTION DE PRICING MONTE CARLO =====
def price_autocall(S0_, r_, q_, sigma_):
    half = N_paths // 2
    np.random.seed(42) # Reproductibilité
    Z = np.random.randn(half, n_steps)
    Z = np.vstack([Z, -Z]) # Variables antithétiques (-Z)

    # Discrétisation du GBM (Mouvement Brownien Géométrique)
    drift = (r_ - q_ - 0.5 * sigma_**2) * dt
    diff = sigma_ * np.sqrt(dt)
    log_increments = drift + diff * Z
    log_S = np.cumsum(log_increments, axis=1)

    S = np.zeros((N_paths, n_steps + 1))
    S[:, 0] = S0_
    S[:, 1:] = S0_ * np.exp(log_S)
    S_rel = S / S0_ # Performance relative

    pv = np.zeros(N_paths)
    autocall_time = np.zeros(N_paths)

    # Évaluation du Payoff (Path-dependent)
    for i in range(N_paths):
        accrued = 0.0 # Mémoire des coupons
        redeemed = False
        for t in range(1, n_steps + 1):
```

```

        accrued += coupon_q
        # Condition 1 : Autocall (Remboursement anticipé)
        if S_rel[i, t] >= autocall_barrier:
            pv[i] = notional * (1 + accrued) * np.exp(-r_ * t *
dt)

            autocall_time[i] = t * dt
            redeemed = True
            break
        # Condition 2 : Paiement du coupon avec mémoire
        elif S_rel[i, t] >= coupon_barrier:
            pv[i] += coupon_q * notional * np.exp(-r_ * t * dt)
            accrued = 0.0 # Reset mémoire

    # Condition 3 : Maturité (si pas autocallé)
    if not redeemed:
        if S_rel[i, -1] >= capital_barrier:
            pv[i] += notional * np.exp(-r_ * T)
        else:
            pv[i] += notional * S_rel[i, -1] * np.exp(-r_ * T)
            pv[i] += accrued * notional * np.exp(-r_ * T)
            autocall_time[i] = T

    return np.mean(pv), pv, autocall_time

# ===== CALCUL DU PRIX ET DES GRECS =====
# Fair Value
price, pv, autocall_time = price_autocall(S0, r, q, sigma)

# Delta (Bump and Reprice)
P_up = price_autocall(S0*1.01, r, q, sigma)[0]
P_dn = price_autocall(S0*0.99, r, q, sigma)[0]
delta = (P_up - P_dn) / (2 * 0.01 * S0) * S0 / notional

# Vega (Bump and Reprice)
P_vol_up = price_autocall(S0, r, q, sigma + 0.01)[0]
P_vol_dn = price_autocall(S0, r, q, sigma - 0.01)[0]
vega = (P_vol_up - P_vol_dn) / 2

# Affichage des résultats en console
print(f"Prix MC (Fair Value) = {price:.4f}% du nominal")
print(f"Delta = {delta:.4f}")
print(f"Vega = {vega:.4f}")

```